HackTwin: The Complete Beginner's Guide

A project explanation for readers with no background in computer architecture, hardware description languages, simulators, digital twins, data analysis or machine learning.

Project: HackTwin — a digital twin framework for Hack CPU HDL designs. Course: 23AID113 Elements of Computing 2, TEAM-14 (team details taken from the title slide of HackTwin_MidTerm_Review.pptx). Document prepared on 22 October 2026 from the repository at `EOC-2 - Copy/hacktwin_run/`.

Contents

1. The simplest explanation
2. Beginner's guide to the background
3. Project requirements and design
4. Complete architecture and data flow
5. Every implemented phase and module
6. Repository map
7. Commands and operating instructions
8. Data, GitHub sources and machine learning
9. Database and generated artifacts
10. Analysis, comparison, dashboard and reports
11. Testing and verification checklist
12. Verified results
13. Incomplete work, limitations and risks
14. Beginner's guided walkthrough
15. Review and viva preparation

> How to read this document. Sections 1 and 2 teach the ideas from zero. Sections 3 to 10 explain how the project is built. Sections 11 to 14 tell you how to run and check it yourself. Section 15 prepares you for a review or viva. Every number in this document was checked against the repository on 22 October 2026. Where something could not be confirmed, it says "not verified in the repository".

1. The simplest explanation

Students in the nand2tetris course build a small computer, called the Hack computer, by writing text files that describe how its electronic parts are connected. To find out whether such a design works, you have to run it in a program called a simulator, which pretends to be the hardware and prints a long table of numbers, one row for every tick of the computer's clock. HackTwin does that work automatically. It runs many designs on many test programs, keeps a careful record of everything that happened, checks the answers, works out where the time went, compares designs fairly, and shows the results on a web page.

One sentence: HackTwin runs Hack CPU designs through the official nand2tetris simulator, records what happens on every clock cycle, stores and analyses those records, compares designs, and shows the findings in a dashboard.

An analogy (this is only an analogy). Imagine a cooking school where every student writes a recipe for the same cake. A tireless assistant bakes every recipe several times in the same oven, writes down exactly what happened minute by minute, tastes each cake to check it is correct, times how long each one took, and then shows the teacher a tidy comparison table. HackTwin is that assistant. The recipes are CPU designs, the oven is the simulator, and the minute-by-minute notes are the traces.

What problem it solves

- Running designs by hand in the simulator is slow and easy to get wrong.
- The simulator's output is a raw table that does not say whether the answer was right, how long the program took, or how one design differs from another.
- Comparing designs fairly needs the same programs, the same inputs, repeated runs and careful bookkeeping. HackTwin does this the same way every time and keeps the evidence.

Who might use it, and why

- Students, to check that their CPU computes correct answers and to see where cycles are spent.
- Teachers or reviewers, to compare many submitted designs on identical tests.
- Anyone experimenting with CPU design changes, to measure whether a change really reduces the number of clock cycles.

What it does not do

- It does not test a real, physical chip. Everything happens in software simulation.
- It does not measure real speed in seconds, the clock frequency, the chip's area, or its power use. The simulator cannot provide these.
- It does not prove that a design would be faster if it were manufactured. "Fewer simulated cycles" is not the same as "faster hardware".
- Its machine-learning predictions are estimates, never measurements.

What "digital twin" means in this project

In industry, a digital twin is a virtual copy of a physical object (for example, a jet engine) that is kept up to date with data measured from the real object. HackTwin borrows the word in a looser, software-only sense: the "object" is a simulated CPU design, and its "twin" is the stored, analysable record of how that design behaved when it ran in the simulator. The twin is updated whenever a design changes or new runs are made. HackTwin is **not** a twin of a physical chip.

2. Beginner's guide to the background

What a computer processor does

A processor (also called a CPU, which stands for Central Processing Unit) is the part of a computer that follows instructions. A program is a list of very simple instructions such as "load the number 2", "add these two numbers" or "jump back to step 6 if the result is not zero". The processor carries them out one after another, extremely quickly.

The Hack computer and the Hack CPU

The Hack computer is a deliberately simple computer designed for the nand2tetris course (from the book *The Elements of Computing Systems* by Nisan and Schocken). Its processor, the Hack CPU, works with 16-bit numbers (numbers stored as 16 binary digits, from –32768 to 32767). It has:

- **Instruction memory (ROM, Read-Only Memory):** holds the program. Each instruction is one 16-digit binary word.
- **Data memory (RAM, Random-Access Memory):** holds numbers the program reads and writes. A memory location is written RAM[0], RAM[1], and so on. Part of RAM is the screen, and one location reads the keyboard.
- **The A register:** a small storage slot inside the CPU. It holds either a number or the address of a RAM location.
- **The D register:** a second slot that holds a data value.
- **The PC (Program Counter):** the address of the instruction the CPU is currently executing. After each instruction it normally moves to the next one, unless the instruction is a jump.

Hack has only two kinds of instruction. An **A-instruction**, written `@5`, puts a number into A. A **C-instruction**, written for example `D=D+A` or `0;JMP`, computes something and may store the result, read or write memory, or jump.

HDL: describing hardware as text

HDL stands for Hardware Description Language. An HDL file does not run like a program. Instead, it **describes how parts are wired together**, a bit like a circuit diagram written as text. For example, one line of the project's CPU design says, in effect, "connect the output of the ALU (Arithmetic Logic Unit, the part that does arithmetic) to the input of the D register". A *design* is a folder of HDL files, one per chip: `CPU.hdl`, `ALU.hdl`, `Memory.hdl` and `Computer.hdl`.

The difference to remember: the **HDL design is the machine**; a **program is what the machine runs**. HackTwin runs the same programs on different machines to compare the machines.

Benchmark programs

A benchmark is a small, fixed test program, run with known inputs, whose correct answer is known in advance. Running the same benchmarks on every design is what makes a comparison fair. HackTwin's main ("core") benchmark suite has 10 programs and 31 input cases. Example: the `mult` program multiplies RAM[0] by RAM[1] and writes the result to RAM[2]; the case `mult-25-40` uses inputs 25 and 40 and expects 1000.

The nand2tetris Hardware Simulator

The Hardware Simulator is the official course tool (a Java program in `nand2tetris/tools/`). It reads HDL files and a test script (a `.tst` file), pretends to be the described hardware, and writes the requested signals to an output file (an `.out` file). If the script names an expected-output file (a `.cmp` file), the simulator also reports whether the output matched. HackTwin runs it in batch mode (without its graphical window), many times.

Clock cycles

Digital hardware moves forward in steps driven by a clock. One step is a **clock cycle**. The single-cycle Hack CPU executes one instruction per cycle. In this project, "cycles" always means simulated clock cycles counted by the simulator, never seconds.

Traces

An execution trace is a table that records the state of the machine on every cycle. Here is a real excerpt (simplified columns) of design 1 running the official Add program, which computes 2 + 3:

cycle	A	D	PC	RAM[0]
0	0	0	0	0
1	2	0	1	0
2	2	2	2	0
3	3	2	3	0
4	3	5	4	0
5	0	5	5	0
6	0	5	6	5

Read it like this: the program loads 2 (cycle 1), copies it into D (cycle 2), loads 3, adds it to D giving 5 (cycle 4), and finally stores 5 into RAM[0] (visible at cycle 6). Source: `hacktwin_run/data/raw_runs/1__ComputerAdd__r0/ComputerAdd.out`.

Correct output, execution time and cycle count

- **Correct output:** the values the program leaves in memory match the expected answer (for example RAM[2] = 1000 for `mult-25-40`).
- **Cycle count (cycles to halt):** the number of simulated cycles until the program reaches its final "stop" loop. Hack programs end with an infinite loop such as `(END) @END 0;JMP`; HackTwin counts cycles until the PC first arrives there.
- **Execution time:** HackTwin also records how many seconds the simulator process took on your laptop ("host wall-clock time"). This includes starting Java and says nothing about hardware speed. It is shown only for information.

A design can be correct and slow, or fast and wrong. Correctness is always checked first.

SQLite: where results are stored

SQLite is a database stored in a single file. A database keeps information in tables (like spreadsheets) that can be searched. HackTwin stores everything in `hacktwin_run/data/hacktwin.db`, so results survive between sessions, can be compared later, and can be traced back to the exact design revision and test configuration that produced them.

Bottleneck analysis and design comparison

A **bottleneck** is a part of the work that takes a large share of the time, like a narrow neck of a bottle that slows the pouring. Bottleneck analysis looks at a trace and asks: which instructions ran most often, which loops took most cycles, how often was memory written? **Design comparison** runs two designs on the same benchmark cases and compares their correctness and cycle counts.

Machine learning, in plain words

Machine learning (ML) means a program learns a pattern from examples instead of being given an exact rule. HackTwin's predictor tries to estimate a design's cycle count for a program without running the HDL simulation. It learns from earlier measured runs. A **prediction error** is the difference between the predicted and the measured number of cycles. The main error measure is the MAE (Mean Absolute Error): the average size of the error, ignoring whether it was too high or too low. An MAE of 50 cycles means predictions were off by about 50 cycles on average. RMSE (Root Mean Squared Error) is similar but punishes large mistakes more. R^2 (R-squared) compares a model with simply guessing the average: 1 is perfect, 0 is no better than the average, and below 0 is worse than the average.

3. Project requirements and design

The original problem and motivation

The mid-term presentation (HackTwin_MidTerm_Review.pptx, slide 2) described the "HDL verification challenge": evaluating CPU designs requires running benchmark programs on every design iteration and capturing performance before the next change, which becomes hard to manage as designs multiply. The current project still addresses exactly this.

Inputs and outputs

Inputs	Outputs
CPU designs: folders of HDL files with a `design.json` description	Per-run records: completed or failed, correct or wrong, cycles to halt
Benchmark programs (`.hack` machine code) with input values and expected answers (`benchmarks/suite.json`)	Stored execution traces and raw simulator output
Official nand2tetris test scripts (`nand2tetris/projects/02` and `05`)	Official test results per design
Optional: public HDL designs and programs from GitHub	Bottleneck findings, comparisons, rankings, recommendations
	Predictor evaluation, web dashboard, generated report (`reports/`)

The user's expected workflow

The web dashboard follows this order: choose a CPU design → choose a benchmark and run it → validate correctness → inspect the trace → analyse bottlenecks → compare designs → review recommendations.

Main design choices and why

Choice	Reason
Use the official simulator, never a home-made replacement	Results must be the course's ground truth.

Measure "cycles to halt" with an instrumented harness	The official test scripts run for a fixed number of cycles, so they cannot show speed differences (see below).
Independent ISA reference emulator (`hack_isa.py`)	Gives expected answers, instruction counts and expected memory writes before any HDL runs. (ISA means Instruction Set Architecture, the rules of what each instruction does.)
SQLite with versioned migrations	One file, no server, safe upgrades of older databases.
Python standard-library web server, plain JavaScript user interface	One command starts everything; no build tools needed; works offline.
Label every claim as measured, inferred or estimated	Prevents overstating what the evidence shows.

How the current project differs from the old presentation

The old presentation is from the mid-term review. Several things changed, and some of its statements are now known to be misleading:

Old presentation said	What the current code shows
Phases 5–8 (predictor, recommendations, dashboard, evaluation) were "work to be done"	All are implemented (see Section 5), with the limitations stated there.
Two designs (1 and 2), described as byte-for-byte identical	Design 2 actually differs from design 1 by one rewritten piece of jump logic. Design 3 is design 2 with different comments. Five new, genuinely different designs were added: 8 HackTwin designs in total.
Design 1 and 2 took 14, 26 and 64 "cycles" on Add, Max and Rect	Those numbers were the number of rows printed by the official test scripts, which are fixed by the scripts (`repeat 14 {...}` etc.). Every design always produces the same counts, so they are not performance measurements.
29 passing unit tests	The current main suite has 136 tests (8 skipped by design) plus integration tests.
A "composite design ranker" using cycle efficiency and register churn	Ranking now uses one documented metric only (geometric-mean speedup), with ties shown as ties.

The review and operating guide (`HACKTWIN_REVIEW_AND_OPERATING_GUIDE.md`, dated 19 October 2026) was accurate about what was implemented at that time, but its statement that designs "measured the same cycle counts" also refers to script row counts. A note has been added to the top of that guide.

4. Complete architecture and data flow

The real flow, as a diagram

```
                                                          +--> analysis (bottlenecks,
comparison, ranking, recommendations) --+
HDL design + benchmark case --> harness + simulator --> raw .out --> parser + checker -
-> SQLite twin store             +--> dashboard / report
        ^
|                       |
GitHub designs (optional)
+--> predictor (ML) ----+
```

In words: a design and a benchmark case go into the simulator through HackTwin's harness; the raw output is parsed and checked; the checked records go into the store; analysis and the predictor read from the store; the dashboard and the report display the results. The official test scripts follow a shorter parallel path (design → simulator → pass/fail → store).

Step-by-step walkthrough

1. **Describe the designs.** In: folders under `designs/` (HDL files plus `design.json`). What happens: `design_catalog.py` reads each folder, and `hdl_inspector.py` parses the HDL, checks that the required chips and connections exist, and computes three fingerprints (explained in Section 9). Out: one design record per folder. Next step uses: the folder path and fingerprints. Success looks like: `python3 design_catalog.py` lists every design with `valid=True`.
2. **Prepare the benchmark cases.** In: `benchmarks/suite.json` and the `.hack` programs. What happens: `benchmark_suite.py` runs each case through the reference emulator (`hack_isa.py`) to get the expected instruction count and memory writes, sets a cycle budget (4 × reference instructions + 64), and computes a configuration fingerprint. Out: case records. Success: `python3 benchmark_suite.py` ends with "All hand-written expected values agree with the reference emulator."
3. **Run the simulator.** In: one design + one case. What happens: `sim_runner.py` copies the design's HDL and HackTwin's harness chip (`harness/TwinComputer.hdl`) into a temporary folder, writes a test script that loads the program, sets the inputs and records the CPU's signals on every cycle, and runs the Hardware Simulator with a time limit. Out: a raw `.out` table. Success: the simulator prints "End of script".
4. **Parse and check the output.** In: the raw table. What happens: `sim_runner.py` turns it into rows, finds the first cycle where the PC reaches the halt address, reads the result values, and compares the sequence of memory writes with the reference emulator. Out: a run record (status, correctness, cycles to halt, details) and the trace. Success: status `succeeded` and correctness `pass`.
5. **Store everything.** In: run records and traces from `experiment_runner.py`, which repeats steps 3–4 for every design, case and repeat (in parallel). What happens: `twin_store.py` writes them to SQLite in one transaction, including failures. Out: rows in `data/hacktwin.db`. Success: `python3 experiment_runner.py` ends with counts such as `by status: {'succeeded': ...}`.

6. **Analyse and compare.** In: stored runs and traces. What happens: `bottleneck_analysis.py` computes hotspots, loops and findings; `comparison_engine.py` compares designs and ranks them; `recommendation_generator.py` turns findings into suggestions and checks whether they were measured. Out: analysis results (computed on demand, not stored). Success: the Bottleneck and Comparison pages show numbers.
7. **Train and evaluate the predictor.** In: valid, correct stored runs. What happens: `cycle_predictor.py` builds one example per (design, case), evaluates six predictors on held-out programs, and saves a model and report. Out: `data/models/cycle_predictor.joblib` and `cycle_predictor_report.json`, also saved in the store. Success: a table of MAE values is printed.
8. **Display and report.** In: the store. What happens: `api_server.py` (started by `app.py`) answers web requests using `hacktwin_service.py`; `ui/` draws the pages; `generate_report.py` writes `reports/`. Out: the dashboard at http://127.0.0.1:8080 and `reports/EXPERIMENT_REPORT.md`. Success: the page loads and the top bar says "simulator found".

The older path (still working)

The original Phase 1–2 path is still present: `run_simulator_batch.py` runs the official Computer tests, `parse_simulator_output.py` writes `data/normalized_runs.jsonl`, and `load_to_store.py` loads it. Its "cycle count" is the script's row count, so the new code labels it `tst_rows` and never compares it.

5. Every implemented phase and module

Status labels used below: **complete** (implemented, tested, used), **partial** (works but covers only part of the goal), **experimental** (works but results must not be relied on), **legacy** (kept, superseded).

Phase 0 — Benchmarks and simulator setup (complete)

Purpose: decide which programs to run and make the simulator callable. Inputs: `benchmarks/suite.json`, `benchmarks/*.hack`, `benchmarks/programs/*.asm`, the nand2tetris tools. Outputs: validated benchmark cases. Main files: `benchmark_suite.py`, `hack_isa.py`, `hacktwin_config.py`. Run: `python3 benchmark_suite.py`. Success: every case is listed and the final line confirms agreement. Common errors: "conformance test … missing" means the nand2tetris folder is not at `../nand2tetris` (set `HACKTWIN_N2T_DIR`). The seven HackTwin-authored programs were assembled both by the official assembler and by `hack_isa.py`; a test confirms the outputs are identical.

Phase 1 — Simulator integration and trace extraction (complete)

Purpose: run designs in the simulator without anyone clicking buttons, and turn the output into data. Inputs: a design folder and a case. Outputs: run results and traces. Main files: `sim_runner.py`, `harness/TwinComputer.hdl`, `experiment_runner.py`; legacy `run_simulator_batch.py` and `parse_simulator_output.py`. Run: `python3 experiment_runner.py` (or `python3 sim_runner.py designs/1 mult-3-5` for one case). Success: "by status: {'succeeded': N}". Common errors: "Simulator command … not found" (Java or tools missing); "too many arguments" (the simulator allows at most 20 recorded columns; HackTwin now stays within this); a timeout means the design did not finish within 120 seconds.

Phase 2 — Digital twin store (complete)

Purpose: keep every result safely with its history. Inputs: run records, traces, design records. Outputs: `data/hacktwin.db`. Main files: `twin_store.py`, `load_to_store.py`, `design_catalog.py`, `summary.py`. Run: `python3 summary.py` (prints design, run and channel counts) or `python3 load_to_store.py --query summary`. Success: non-zero counts. Common errors: "database is locked" if another program holds a long write (wait and retry). The store upgrades older databases automatically (Section 9).

Phase 3 — Bottleneck analysis (complete, heuristic by nature)

Purpose: show where cycles are spent. Inputs: one stored trace. Outputs: hotspots, loops, instruction mix, register and memory activity, and labelled findings. Main file: `bottleneck_analysis.py` (the original `analyze_run_telemetry` for legacy runs and the new `analyze_program_trace`). Access: the Bottleneck analysis page, `/api/runs/<run_id>/analysis`, or the legacy CLI `python3 bottleneck_analysis.py --run-id 1__ComputerAdd__r0`. Findings are typed as observation, heuristic, confirmed problem or trace-quality warning; a frequent event is never automatically called a defect.

Phase 4 — Comparison and ranking (complete)

Purpose: compare designs fairly. Inputs: stored runs. Outputs: per-case speedups, aggregate results, rankings. Main file: `comparison_engine.py` (`compare_program_designs`, `rank_program_designs`; the older `compare_two_designs` and `rank_designs` remain for legacy runs). Access: Design comparison page, `/api/compare`, `/api/ranking`, legacy CLI `python3 comparison_engine.py --designs 1,2`. Exact rules are in Section 10.

Phase 5 — Performance prediction (implemented; results mixed — see Section 8)

Purpose: estimate cycle counts without HDL simulation. Main files: `cycle_predictor.py` (current), `ml_performance_predictor.py` (earlier model, still used by the old dashboard), `performance_predictor.py` (legacy heuristic, not validated), `collect_github_workloads.py`, `build_github_training_data.py`. Run: `python3 cycle_predictor.py --train`. Status: the calibrated instruction-count predictor is accurate for designs it has measured; the text-only ML models are weak and are labelled experimental.

Phase 6 — Recommendations and mutation (complete; recommendations are suggestions)

Purpose: turn findings into concrete next experiments. Main files: `recommendation_generator.py` (`recommend_for_program_run` and the older `RecommendationEngine`), `mutation_engine.py` (creates a design variant from a template; it produced design 3). Access: Recommendations page, `/api/designs/<id>/recommendations`, `/api/runs/<id>/recommendations`, `python3 recommendation_generator.py --run-id …`, `python3 mutation_engine.py --history`. Every recommendation has an evidence status: confirmed problem, heuristic, experimentally verified, experimentally refuted, or software suggestion. "Verified" is used only when the suggested design was actually simulated on the same case and used fewer cycles.

Phase 7 — Dashboard and API (complete)

Purpose: let a person explore everything without the command line. Main files: `app.py`, `api_server.py`, `hacktwin_service.py`, `ui/`. Run: `python3 app.py --port 8080` and open http://127.0.0.1:8080. The API (Application Programming Interface: web addresses that return data instead of pages) is listed in Section 7. Simulations started from the page run as background "jobs" so the page stays responsive. The older dashboard (`dashboard_server.py`, `dashboard/`) still works on another port; the Streamlit prototype (`streamlit_app/`) is legacy.

Phase 8 — Evaluation and report (complete)

Purpose: produce reproducible evidence. Main files: `generate_report.py` (current), `evaluation_study.py` (earlier report, `evaluation_report.md`), `demo.py` (end-to-end from an empty database). Run: `python3 generate_report.py`. Output: `reports/EXPERIMENT_REPORT.md`, `reports/experiment_results.json` and four SVG charts.

GitHub HDL dataset (complete; covers part of what was found)

Main file: `github_hdl_dataset.py`. Described fully in Section 8.

Important supporting modules

Module	What it does
`hack_isa.py`	Assembler (text to machine code), disassembler (machine code to text) and the reference emulator used as an independent answer key.
`hdl_inspector.py`	Reads HDL files, checks them, and computes the source, normalized and netlist fingerprints.
`hacktwin_config.py`	All file locations and environment variables in one place.
`design_catalog.py`	Finds designs and registers them in the store.
`hacktwin_service.py`	Prepares the data for each dashboard page.

6. Repository map

Item	Type	Edit, run or leave alone	Used by
`hacktwin_run/README.md`	Documentation	Read	You
`app.py`, `api_server.py`	Source (entry point)	Run `app.py`	Dashboard
`designs/<id>/`	Input data (HDL + `design.json`)	Edit to create designs	Catalog, simulator
`benchmarks/suite.json`, `benchmarks/programs/`	Input data	Edit to add cases	Benchmark suite

`harness/TwinComputer.hdl`	Source (HDL)	Leave alone	Simulator runs
`*.py` in `hacktwin_run/`	Source	Run the scripts in Section 7	Each other
`test_*.py`, `test_support.py`	Tests	Run	Unit tests
`ui/`	Source (web pages)	Edit only for UI changes	Browser
`data/hacktwin.db`	Database (generated)	Inspect read-only; regenerate with scripts	Everything
`data/backups/`	Database backup	Leave alone	Recovery
`data/raw_runs/`, `data/normalized_runs.jsonl`	Generated output (legacy path)	Regenerate	Legacy loader
`data/external/`	Generated (GitHub downloads, API cache, imported designs)	Leave alone	GitHub pipeline
`data/github_workloads/`	Input data (collected programs, licences, labels)	Leave alone	Predictors
`data/models/`	Model artifacts and evaluation reports	Regenerate by training	Predictor page
`examples/example_twin.db`	Labelled example database	Use for exploring	Dashboard
`reports/`	Generated report	Regenerate	Reviewers
`docs/AUDIT.md`	Documentation	Read	Reviewers
`dashboard/`, `dashboard_server.py`, `streamlit_app/`	Legacy source	Leave alone	Old dashboard
`../nand2tetris/`	Third-party tools and course files	Leave alone	Simulator

7. Commands and operating instructions

Run every command from the `hacktwin_run` folder unless stated otherwise. Open a terminal and type:

```
cd "/Users/cow/Desktop/EOC-2 - Copy/hacktwin_run"
```

Set-up

```
python3 -m pip install -r requirements.txt
```

What it does: installs numpy, scikit-learn and joblib. Success: no error. Check with `python3 -c "import numpy, sklearn, joblib; print('ok')"`. If it fails: make sure `python3` is installed and you have internet access.

```
java -version
```

What it does: confirms Java is installed (the simulator needs it). Success: a version line such as `openjdk version "11.0.32"`. If it fails: install Java.

Start and stop the dashboard

```
python3 app.py --port 8080
```

Success: "HackTwin running at http://127.0.0.1:8080". Open that address in a web browser. To stop it, press Ctrl+C in the terminal. If it fails with "Address already in use", another program uses port 8080: choose another, e.g. `--port 8090`.

```
HACKTWIN_DB=examples/example_twin.db python3 app.py --port 8080
```

Opens the bundled example database instead (works without Java). A yellow banner marks it as example data.

Run experiments

```
python3 experiment_runner.py
```

Runs every design on the core suite with 3 repeats plus the official tests. Success: "Experiment … finished" and status counts. Runs that already succeeded are skipped, so a second run is quick. Useful options: `--designs 1,jumpfold-8`, `--repeats 2`, `--suite workloads`, `--cases mult-25-40`, `--no-conformance`, `--force` (rerun even successful runs).

```
python3 sim_runner.py designs/1 mult-25-40
```

Runs one case on one design and prints the full result, including `"cycles_to_halt": 490`.

```
python3 demo.py
```

Rebuilds everything from an empty database (`data/demo.db`) in about 30 seconds and writes `reports/demo/`. It prints the command to open its dashboard.

Inspect designs, benchmarks and the store

```
python3 design_catalog.py
python3 hdl_inspector.py designs/2 designs/3
python3 benchmark_suite.py
python3 summary.py
python3 load_to_store.py --query summary
```

`hdl_inspector.py` with two folders prints the difference level; for designs 2 and 3 it prints `difference level: comments_or_whitespace_only`.

Analysis, comparison and recommendations (command line)

```
python3 bottleneck_analysis.py --run-id 1__ComputerAdd__r0
python3 comparison_engine.py --designs 1,2
python3 comparison_engine.py --rank
python3 recommendation_generator.py --run-id 1__ComputerAdd__r0
python3 mutation_engine.py --history
```

These command-line tools are the original Phase 3, 4 and 6 tools. They work on the legacy official-test runs, whose "cycles" are script row counts, so their comparisons always tie. Use the dashboard for the current measurements.

Predictor and reports

```
python3 cycle_predictor.py --train
python3 cycle_predictor.py --predict --design jumpfold-8 --case mult-25-40
python3 generate_report.py
python3 ml_performance_predictor.py --train
```

The last command retrains the earlier model; it needs the GitHub workload labels in `data/github_workloads/`.

GitHub dataset

```
GITHUB_TOKEN=your_token python3 github_hdl_dataset.py discover --max-pages 1
GITHUB_TOKEN=your_token python3 github_hdl_dataset.py ingest --max-repos 300
python3 github_hdl_dataset.py validate --repeats 2
python3 github_hdl_dataset.py report
```

A token is optional but raises GitHub's limit from 60 to 5,000 requests per hour. If you use the GitHub command-line tool, `GITHUB_TOKEN=$(gh auth token)` supplies one. The token is never written to disk.

Legacy official-test pipeline

```
python3 run_simulator_batch.py --sim-cmd ../nand2tetris/tools/HardwareSimulator.sh --
repeat 2
python3 parse_simulator_output.py
python3 load_to_store.py
python3 evaluation_study.py
```

Tests

```
python3 -m unittest discover
HACKTWIN_INTEGRATION=1 python3 -m unittest test_integration_simulator
```

The first runs 136 fast tests (no simulator, no internet). The second runs the real simulator (about 10 seconds).

API endpoints (verified in `api_server.py`)

Method and path	Returns
GET `/api/meta`	Schema version, simulator status, glossary, metric definitions
GET `/api/overview`	Everything on the Overview page
GET `/api/designs`, `/api/designs/<id>`	Design list; one design's details
GET `/api/designs/diff?a=1&b=2`	HDL text differences and difference level
GET `/api/designs/<id>/files/<name>`	One HDL file's text
GET `/api/designs/<id>/recommendations`	Recommendations across that design's cases
GET `/api/benchmarks`	Benchmarks, cases and run counts
GET `/api/runs?design=&benchmark=&case=&status=&scope=`	Runs (scope: current, historical, all)
GET `/api/runs/<run_id>`	One run with repeat consistency
GET `/api/runs/<run_id>/trace?filter=&value=&offset=&limit=`	A window of the trace (max 500 rows)
GET `/api/runs/<run_id>/pc`	PC over time, compressed
GET `/api/runs/<run_id>/raw`	Raw simulator output lines
GET `/api/runs/<run_id>/analysis`, `/recommendations`	Bottleneck analysis; run recommendations
GET `/api/compare?baseline=1&candidates=a,b&benchmarks=`	Comparison
GET `/api/ranking`	Ranking
GET `/api/predictor`; POST `/api/predictor/predict`	Predictor report; a prediction for `{design_id, case_id}`
GET `/api/dataset`, `/api/experiments`	GitHub funnel and candidates; experiments and jobs
POST `/api/jobs`; GET `/api/jobs/<id>`	Start a simulation job; check its progress

8. Data, GitHub sources and machine learning

Two different uses of GitHub

HackTwin uses public GitHub repositories (online project folders) in two separate ways. A **GitHub dataset** here means a collection of files downloaded from such repositories together with records of exactly where each file came from.

(a) Extra test programs. `collect_github_workloads.py` downloads archives from six repositories pinned to exact versions ("commits") and checked for permissive licences: Gunasekare/nand2tetris (MIT), ArhanChaudhary/NAND (MIT), y1hao/Nand2Tetris (MIT), aalhour/Assembler.hack (MIT), appleweiping/nand2tetris (MIT) and AdamJelley/Nand2Tetris (Apache-2.0). It keeps valid Hack programs, removes duplicates and saves the licences. Result on record: 50 unique programs

(`data/github_workloads/source_manifest.json`). Of these, 19 halt cleanly in the reference emulator for all three input profiles and are used as the "GitHub workload" benchmark suite.

(b) Other people's CPU designs. `github_hdl_dataset.py` searches GitHub, finds folders containing `CPU.hdl`, downloads only `.hdl` files at an exact commit, and tests them. Rules: files larger than 256 KiB are refused; at most 20 folders per repository are taken; HDL using the `BUILTIN` keyword is rejected for safety; no code from the repositories is ever run; missing `Memory.hdl`/`Computer.hdl` are supplied from HackTwin's baseline and this is recorded. Each candidate moves through stages: discovered → retrieved → structurally valid → simulator-compatible → correct → complete. Duplicates (identical files or identical wiring) stop at "structurally valid".

Results recorded on 21 October 2026:

Stage	Count
Repositories found by search	1,346
Repositories retrieved (bounded run; 1,005 still pending)	339
Repositories containing a CPU.hdl	234
Candidate CPU folders	344
Structurally valid	296
Duplicates	60
Unique and structurally valid	236
Load in the simulator	213
Correct on all tests and complete	208

Licences of the repositories with a CPU.hdl: 189 none detected, 31 MIT, 8 GPL-3.0, 2 Unlicense, 2 CC0-1.0, 1 BSD-3-Clause, 1 GPL-2.0. Files without a licence are only kept locally for analysis.

What a training example is

For the current predictor (`cycle_predictor.py`), one example is one (design, benchmark case) pair, with the measured cycles to halt as the answer to learn. Repeat runs are merged, because they are identical. Only HackTwin's own 8 designs are used by default; imported designs are excluded because nearly all share the baseline's timing and would flood the data with copies. Current dataset: 704 examples, 8 designs, 88 cases, 28 distinct programs.

Features (the clues the model sees)

- **Static features:** counts taken from the program text, such as the number of instructions, jumps and backward jumps, plus the first two input values.
- **Dynamic features:** counts from running the program in the reference emulator (not the HDL simulator): instructions executed, memory reads and writes, branches taken, and "foldable jumps".

No design name is given to the models.

Training and evaluation

Evaluation uses held-out programs: the examples are split into groups by program, and each time the model is trained on some programs and tested on programs it has never seen (GroupKFold, up to 5 folds). This prevents "cheating" by seeing another input of the same program during training. Six predictors are compared:

Predictor	Average MAE (cycles), over the 8 designs
Calibrated instruction model (cycles = a × instructions + b × foldable jumps, fitted per design)	0.60
ExtraTrees (a tree-based ML model) with static + dynamic features	53.7
Instruction count with 1 cycle per instruction assumed	60.1
ExtraTrees with static features only	403.5
Median guess (the baseline every model must beat)	490.0
Ridge regression (a straight-line model) with static features only	1,966

Source: `data/models/cycle_predictor_report.json`, created 21 October 2026.

What these numbers mean and how far to trust them

- The calibrated instruction model is almost exact, but only because it is calibrated with measurements of the same design. Tested on a design it has not seen (leave-one-design-out), it is badly wrong for the two-cycle design.
- Models that only read the program text are barely better than the median guess, or worse. They should not be trusted.
- Five of the eight designs have identical timing, so their results are not independent evidence.
- The dataset is small (28 programs). Predictions for very different programs are not supported by evidence.

The earlier model

`ml_performance_predictor.py` (described in the operating guide) trained ExtraTrees on 103 measured examples from 21 programs, on design 1 only. Its grouped evaluation gave MAE 439.5 cycles against 513.2 for the median guess, with R^2 of −0.0856 (worse than guessing the average). The guide correctly calls it exploratory. Source: `data/models/hacktwin_cycle_model.validation.json`.

Is the model connected to the dashboard?

Yes. The current predictor is shown on the dashboard's Performance predictor page (`/api/predictor`, `POST /api/predictor/predict`), which reads the saved report and model. The earlier model is used by the old dashboard (`dashboard_server.py`, route `/api/predict`).

9. Database and generated artifacts

Tables

Table	One row means	Important fields
`schema_migrations`	One applied upgrade of the database layout	version (now 3), description
`designs`	One CPU design	design_id, hdl_source_hash, normalized_hash, netlist_hash, parent_design_id, kind, provenance
`benchmarks`	One benchmark program	benchmark_id, suite, program_sha256
`benchmark_cases`	One program + input set	case_id, inputs, expected, max_cycles, config_hash
`runs`	One simulator run	run_id, design_id, case_id, repeat_index, mode, status, correctness, cycles_to_halt, run_hdl_hash, config_hash, failure_reason
`traces`	The trace of one run	columns, rows (compressed), raw simulator output, sha256
`channels`	One output file of a legacy run	channel_name, cycle_count
`conformance_results`	One official test on one design revision	test_name, outcome, message
`experiments`	One batch of runs	title, configuration, summary
`jobs`	One simulation started from the dashboard	status, progress, result, error
`mutations`	One variant made by the mutation engine	parent, child, type
`gh_repositories`, `gh_candidates`	One GitHub repository; one CPU folder in it	licence, commit, stage, status, errors, duplicate_of
`model_reports`	One predictor evaluation	report
`store_meta`	A label such as "example dataset"	key, value

`run_summary` is a view (a saved query) that joins runs with designs.

From a simulation to stored rows

The run identifier encodes what was run, for example `jumpfold-8__mult-25-40__hb6529dedc1a8eb1__r0`: design, case, the first characters of the design's revision fingerprint and of the case configuration fingerprint, and the repeat number. Re-running the same thing replaces the same row instead of making a duplicate. A changed HDL file gives a new fingerprint, so new rows are created and the old ones stay as history ("not current"). Failed runs are stored with their error message and no cycle count.

Design identity and revisions

- **Source hash:** a fingerprint of the files' exact bytes. Any edit, even a comment, changes it.
- **Normalized hash:** the same with comments and spaces removed.
- **Netlist hash:** a fingerprint of how parts are wired, ignoring part order and wire names. Equal netlist hashes strongly suggest identical hardware (it is not a mathematical proof).

Lineage

Each design may name a parent (`parent_design_id`), for example `jumpfold-1` → `jumpfold-8` → `1`. Lineage records history; it does not mean the child is better.

Generated artifacts (all can be regenerated)

Artifact	Made by
`data/hacktwin.db`	experiment_runner.py, github_hdl_dataset.py, load_to_store.py
`data/models/cycle_predictor.*`	cycle_predictor.py --train
`reports/`	generate_report.py
`data/demo.db`, `reports/demo/`	demo.py
`examples/example_twin.db`	demo.py --example
`data/external/`	github_hdl_dataset.py

Inspecting the store safely

Use read-only queries. For example:

```
sqlite3 -readonly data/hacktwin.db "SELECT design_id, kind FROM designs WHERE kind !=
'external';"
sqlite3 -readonly data/hacktwin.db "SELECT status, correctness, COUNT(*) FROM runs
WHERE mode='program' GROUP BY 1, 2;"
```

The pre-upgrade database is saved in `data/backups/hacktwin_pre_v2_20261021.db`.

10. Analysis, comparison, dashboard and reports

Overview page

Question: what is in the store and what does it show? You see counts of designs (with distinct source and netlist fingerprints), benchmarks, runs (current and historical), the latest activity time, the ranking chart, the best design for a chosen benchmark, the baseline's bottleneck signals, official test totals, the GitHub funnel and the predictor summary. All values come from the store. Safe conclusion: which designs are correct and which used fewer simulated cycles. Not safe: anything about real chip speed.

CPU designs page

Question: what is each design, and is it really different? You see each design's kind, parent, fingerprints, official test results, coverage, speedup and history, its HDL files, and a "difference vs

baseline" explanation that separates text differences from wiring differences.

Simulation workbench

Question: what happens if I run this design on this case? You choose a design, a benchmark and cases, then press Run. A progress bar shows the background job. Each run's page shows status, cycles to halt, CPI (cycles per instruction), expected versus actual values (and whether each came from a direct memory probe or was reconstructed from the observed memory writes), architectural equivalence (same memory writes in the same order as the reference model), and repeat consistency.

Trace explorer

Question: what exactly happened, cycle by cycle? You see the PC plotted over time, a bar chart of cycles spent at each address, and a table of 100 cycles at a time with filters (memory writes, reads, jumps, a given PC or address, or text). Observed columns come straight from the simulator; the decoded instruction text and event labels are inferred by HackTwin.

Bottleneck analysis

Question: where do the cycles go? You see cycles to halt, CPI, foldable jumps, folded jumps, labelled findings with evidence, instruction-class shares, hottest addresses, loops, and register and memory activity. Only cycles before the program halts are counted.

Design comparison: the exact rules

1. Only runs that completed, halted, produced correct results, belong to the design's current revision and use the case's current configuration are compared.
2. **Speedup = baseline cycles ÷ candidate cycles** for each case. Above 1 means fewer cycles.
3. **Cycle reduction = (baseline – candidate) ÷ baseline × 100%.**
4. The aggregate is the geometric mean of the per-case speedups (a kind of average suited to ratios, in which every case counts equally).
5. Cases that failed or are missing are listed, never counted as zero.
6. If the designs are identical or only differ in comments, the page says the comparison only validates the pipeline.

Ranking: designs are ordered by geometric-mean speedup over all 31 core cases. A design wrong on any case is not ranked. Equal values share a rank. No other numbers are mixed in.

Are the designs genuinely different, and is any advantage measurable? Yes for some. Designs 2, `alu-xor-negate` and `branch-table` have different wiring but exactly the same cycles on all 31 cases. Design 3 has the same wiring as design 2. `jumpfold-8` and `jumpfold-1` use measurably fewer cycles on loop-heavy cases. `multicycle-2phase` uses exactly twice as many.

Recommendations, predictor and datasets pages

Recommendations: suggestions with evidence and status (Section 5). Performance predictor: the evaluation tables and a "try a prediction" form that also shows the measured value when one exists.

Datasets & experiments: the GitHub funnel, rejection reasons, licences, candidate and repository tables, experiments and jobs. Glossary: definitions of the terms.

The report

`reports/EXPERIMENT_REPORT.md` is regenerated from the store and contains the tables of Section 12, the GitHub results, the predictor evaluation, the unsuccessful runs and the limitations.

11. Testing and verification checklist

Unit tests (fast; no simulator or internet)

Action	Expected	Verifies	If it fails
`python3 -m unittest discover`	"Ran 136 tests … OK (skipped=8)"	All modules' logic with small synthetic data	Read the first FAIL line; it names the test file
`python3 -m unittest test_hack_isa`	OK	Assembler matches official output; emulator rules	A changed program file or assembler
`python3 -m unittest test_hdl_inspector`	OK	HDL parsing, fingerprints, design checks	An HDL file was edited incorrectly
`python3 -m unittest test_store_v2`	OK	Database upgrades, transactions, history	Database code changes
`python3 -m unittest test_comparison_v2`	OK	Speedup, ties, failures, rankings	Comparison rules changed
`python3 -m unittest test_github_dataset`	OK	Ingestion stages, duplicates, safety, token handling (GitHub is simulated)	Ingestion code changes
`python3 -m unittest test_api_server`	OK	API answers, errors, concurrent requests	Port or server code issues

The 8 skipped tests are the integration tests, which only run when asked.

Integration tests (real simulator)

```
HACKTWIN_INTEGRATION=1 python3 -m unittest test_integration_simulator
```

Expected: "Ran 8 tests … OK (skipped=1)". Verifies: real simulator runs give the recorded cycle counts (for example 490 for design 1 on `mult-25-40`), the baseline matches the reference model on every case, repeats are identical, official tests pass or fail as expected, broken and wrong designs

are detected, and a small experiment works end to end. The skipped one queries GitHub live; enable it with `HACKTWIN_NETWORK=1`.

Phase-by-phase manual checks

Phase	Action	Expected
0	`python3 benchmark_suite.py`	Last line: values agree with the reference emulator
1	`python3 sim_runner.py designs/1 mult-25-40`	`"status": "succeeded"`, `"correctness": "pass"`, `"cycles_to_halt": 490`
2	`python3 summary.py`	Non-zero design and run counts
3–6	Open the dashboard's Bottleneck, Comparison and Recommendations pages	Numbers and findings appear
5	`python3 cycle_predictor.py --train`	A table of MAE values
7	`python3 app.py --port 8080`	Page loads; top bar says "simulator found"
8	`python3 generate_report.py`	"Wrote … EXPERIMENT_REPORT.md"

End-to-end check

Run `python3 demo.py`, then start the dashboard with the command it prints, and confirm the Overview shows the demo designs and the ranking. This exercises the whole path from HDL to dashboard. Note that the unit tests do not run the simulator; only the integration tests and the demo do.

12. Verified results

All values below are **measurements** from the store, generated into `reports/EXPERIMENT_REPORT.md` on 21 October 2026, unless marked otherwise.

Size of the evidence

Item	Value
HackTwin designs	8
Imported GitHub designs in the store	215 (208 correct and complete)
Core benchmark programs / input cases	10 / 31
GitHub workload programs	19 (57 cases)
Repeats per core case	3 for the main experiment (2 for GitHub designs)
Run-to-halt simulation runs stored	14,682
Legacy official-test runs	48
Official-test and static-validation results recorded	1,547

Correctness and agreement

All 8 HackTwin designs produced the correct result on all 31 core cases, every repeat was byte-for-byte identical, and every design's memory writes matched the reference model. Official tests: every

design passed all six, except `multicycle-2phase`, which passes only the ALU test because the others check timing cycle by cycle (expected, since it takes two cycles per instruction).

Cycle counts (selected cases)

Case	Baseline (1)	jumpfold-8	jumpfold-1	multicycle-2phase
add	6	6	6	12
max-3-5	12	12	12	24
mult-25-40	490	451	451	980
fib-20	456	437	437	912
bubblesort-8-rev	1,212	1,179	1,191	2,424
div-1000-3	4,008	3,676	3,676	8,016
rect-16	218	218	218	436
Total, all 31 cases	9,766	9,136	9,156	19,532

Designs 2, 3, `alu-xor-negate` and `branch-table` match the baseline on every case.

Comparison summary (calculated from the measurements)

Candidate vs baseline	Fewer / equal / more cycles	Geometric-mean speedup	Total cycle reduction
2, 3, alu-xor-negate, branch-table	0 / 31 / 0	1.0000	0.00%
jumpfold-8	16 / 15 / 0	1.0324	6.45%
jumpfold-1	16 / 15 / 0	1.0316	6.25%
multicycle-2phase	0 / 0 / 31	0.5000	–100.00%

What this demonstrates and what it does not

It demonstrates that the measurement pipeline detects real cycle differences (the two-cycle design is exactly 2×), that jump folding saves cycles on loops ending in an unconditional jump (no saving on `rect`, whose loop ends differently), and that rearranging logic alone did not change any cycle count. All 208 correct GitHub designs had exactly the baseline's cycle counts. It does **not** demonstrate faster real hardware: clock speed, area and power were not measured, and jump folding adds hardware that could slow the clock.

Model results

See Section 8. These are **estimates evaluated against measurements**, not measurements themselves.

13. Incomplete work, limitations and risks

Item	Evidence	Effect	Sensible next step

No real timing, area or power	Simulator only counts cycles	Cannot claim faster chips	Use a synthesis tool for timing/area
GitHub coverage partial	339 of 1,346 repositories retrieved	Dataset may not represent all designs	Run `ingest` again; it resumes
Imported designs give no timing variety	All 208 have baseline timing	They test correctness, not speed	Seek designs with different timing
Predictor depends on calibration	Leave-one-design-out errors are large	Cannot predict a new kind of design	Measure a design before predicting
Small, partly duplicated ML data	28 programs; 5 designs share timing	Weak evidence for ML	More varied programs and designs
Limited memory observation	At most 11 RAM cells probed per run	Some results checked from the write record	Accept; labelled in the UI
Some CPUs not observable by official tests	Designs without built-in A/D/PC registers	Tests reported "not applicable"	None needed; noted
Memory.tst excluded	It waits for keyboard input	Memory chip not tested by that script	A headless memory test
Legacy tools report fixed "cycles"	Script row counts	Can confuse new users	Use the new dashboard
Jump folding assumes unchanging program memory	Design notes	Fine for Hack's ROM	None
Environment dependence	Needs Java, the tools and curl	Runs fail if missing	Follow Section 7 set-up
Old database scripts	`evaluation_study.py` used to overwrite design fingerprints; fixed on 22 October 2026	Older copies could mark runs as stale	Use the current file
No pipelined CPU	Not implemented	No pipeline results	Possible future design

14. Beginner's guided walkthrough

1. Open a terminal and go to the project: `cd "/Users/cow/Desktop/EOC-2 - Copy/hacktwin_run"`.
2. Run `python3 app.py --port 8080` and open http://127.0.0.1:8080. Notice the top bar: "Store schema v3 · simulator found" means the database and simulator are ready.
3. On **Overview**, find the ranking chart. Notice that `jumpfold-8` is above 1.0, several designs sit exactly at 1.0 (a tie), and `multicycle-2phase` is at 0.5.

4. Click **CPU designs**, then design `3`. Open "Summary" and read the difference from the baseline. Then try the HDL diff of designs 2 and 3: only comments differ. This shows why a different file is not necessarily a different machine.
5. Click **Simulation workbench**. Choose design `jumpfold-8`, benchmark "Multiply", leave all cases selected, and press **Run simulation**. Watch the progress bar finish.
6. Click the row `mult-25-40`. Notice: cycles to halt 451, ISA instructions 490, CPI about 0.92, every expected value marked "match", and "architecturally equivalent". This means the design computed the same answer with the same memory writes, in fewer cycles.
7. Click **Trace explorer**. In the PC chart, the line jumps from 16 straight back to 6 without visiting 17: the hardware skipped the jump instruction. That is jump folding, seen directly in the trace.
8. Click **Bottleneck analysis**. Read the finding "39 jumps were folded by the hardware" and the loop that covers about 99% of cycles.
9. Click **Design comparison**, select `jumpfold-8` and `multicycle-2phase`, and press Compare. Read each conclusion box; note the sentence that fewer cycles is not evidence of faster hardware.
10. Click **Recommendations** for design `1`. The jump-folding suggestion is marked "experimentally verified" because `jumpfold-8` was actually measured on the same cases.
11. Stop the server with Ctrl+C.

15. Review and viva preparation

Short summary (presentation-ready)

HackTwin automatically runs Hack CPU designs written in HDL through the official nand2tetris simulator, records every clock cycle, checks the results against an independent reference model, stores everything in a SQLite database, and compares designs on identical benchmarks. Across 8 designs and 31 test cases, every design was correct; jump folding saved 6.45% of simulated cycles, a two-cycle design took exactly twice as long, and logic rewrites changed nothing. It measures simulated cycles only, not real hardware speed.

Longer explanation to say aloud

"In our course we build a computer, the Hack computer, by describing its chips in a hardware description language. Testing a design means running it in a simulator that prints one row per clock cycle, which is hard to read and compare by hand. HackTwin automates this. It takes CPU designs and benchmark programs with known answers, runs every combination in the official simulator several times, and records a trace of what happened. It checks each answer two ways: against the expected value and against an independent model of the Hack instructions. Everything goes into a SQLite database, together with the design's fingerprints and history, so results can always be traced back. On top of that we analyse where cycles are spent, compare designs fairly using only correct runs of the same programs, and show it all in a web dashboard. We also built eight designs: some only rewire logic and turned out to take exactly the same cycles, one takes two cycles per instruction and is exactly twice as slow, and one skips repeated jump instructions and saves about six and a half percent of cycles. We imported more than two hundred designs from GitHub; all correct ones had the same timing as our baseline. A machine-learning predictor estimates cycle

counts; it works well only after it is calibrated with measurements of the same design. And importantly, all of this is simulation: we make no claim about real chip speed, area or power."

Likely viva questions

1. **What is HackTwin in one sentence?** A tool that runs Hack CPU designs through the official simulator, records each clock cycle, stores and analyses the records, compares designs and shows the results.
2. **Why is it called a digital twin?** Because it keeps an up-to-date, analysable record of how each simulated design behaves. It is a software sense of the word, not a twin of a physical chip.
3. **What is HDL, and how is it different from a program?** HDL describes how hardware parts are wired. A program is a list of instructions the hardware runs. We run the same programs on different hardware designs.
4. **What does the simulator give you, and what is missing from it?** One row of signals per cycle and a pass/fail against an expected file. It does not say how many cycles a program needed to finish or how designs compare.
5. **Why weren't the official test scripts enough to compare speed?** They run for a fixed number of cycles (for example `repeat 14`), so every design prints the same number of rows. We added a harness that measures cycles until the program actually finishes.
6. **How do you know an answer is correct?** We compare the final memory values with the expected answer and the sequence of memory writes with an independent reference emulator.
7. **What is a trace, and what do you record?** A cycle-by-cycle table: PC, instruction, A and D registers, memory address, values read and written, and selected RAM cells.
8. **What is stored in the database?** Designs and their fingerprints and lineage, benchmark cases, every run including failures, compressed traces, official test results, experiments, jobs, GitHub records and model reports.
9. **How do you tell whether two designs are really different?** Three fingerprints: bytes, text without comments, and wiring (netlist). Design 3 differs from design 2 only in comments.
10. **What is a bottleneck in your analysis?** A part of the run taking a large share of cycles, such as a loop. We label findings as observations or heuristics and do not call frequent events defects.
11. **How is speedup defined?** Baseline cycles divided by candidate cycles on the same case. The overall figure is the geometric mean over cases where both designs are correct.
12. **Which design is best, and is it really faster?** `jumpfold-8` needs the fewest simulated cycles (geometric-mean speedup 1.0324). We cannot say it is a faster real chip, because clock speed and area were not measured.
13. **Why did the two-cycle design fail the official tests if it is correct?** Those tests compare signals cycle by cycle against the standard single-cycle timing. Its answers are correct but its timing is different by design.
14. **What did you take from GitHub?** 50 programs from six licensed repositories, and 344 CPU folders from public repositories, of which 208 passed every test. We fetched only HDL files by

exact commit and never ran their code.

15. **What does your ML model predict, and is it accurate?** It predicts cycles to halt. The calibrated instruction model is accurate (MAE 0.6 cycles) for designs already measured; models using only the program text are weak, and the earlier model had R^2 below zero.
16. **How did you avoid data leakage in evaluation?** By grouping examples by program and testing only on programs never seen in training.
17. **What are the main limitations?** Simulation only; partial GitHub coverage; little timing variety among imported designs; a small ML dataset; limited memory probes.
18. **What would you do next?** Process the remaining repositories, try a genuinely pipelined CPU, and use a synthesis tool to estimate timing and area.

Literature review (from the old presentation)

The old presentation's literature-review slide is an image of a table, which is readable. It lists the following six works, reproduced here with shortened descriptions (not independently re-checked against the original papers):

Paper	Authors	Year	Main contribution (as stated in the old slide)
Digital Twin in Manufacturing: A Categorical Literature Review and Classification	Werner Kritzinger, Matthias Karner, Georg Traar, Jan Henjes, Wilfried Sihn	2018	Classifies digital-twin concepts and relates physical systems, virtual representations and data exchange.
A Survey on Digital Twin: Definitions, Characteristics, Applications, and Design Implications	Barbara R. Barricelli, Elena Casiraghi, Daniela Fogli	2019	Reviews digital-twin definitions, characteristics, applications and design considerations.
Digital Twin: Enabling Technologies, Challenges and Open Research	Aidan Fuller, Zhong Fan, Charles Day, Chris Barlow	2020	Reviews enabling technologies and open challenges, including simulation, data management, analytics and prediction.
Characterising the Digital Twin: A Systematic Literature Review	David Jones, Chris Snider, Aydin Nassehi, Jason Yon, Ben Hicks	2020	Systematically reviews digital-twin definitions, characteristics and implementations.
The gem5 Simulator	Nathan Binkert et al.	2011	A configurable computer-system simulator for processor and architecture research.
McPAT: An Integrated Power, Area, and Timing Modeling Framework for Multicore and Manycore Architectures	Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, Norman P. Jouppi	2009	A framework for modelling processor power, area and timing.

How HackTwin relates: it borrows the digital-twin idea from the first four works but applies it to simulated designs. Like gem5 it is simulation-based, but it uses the course's own simulator. Unlike McPAT, it does not model power, area or timing.

Glossary

Term	Meaning
A register	A storage slot in the Hack CPU holding a number or a memory address.
ALU	Arithmetic Logic Unit: the part of the CPU that does arithmetic and logic.
API	Application Programming Interface: web addresses that return data for programs.
Baseline	The reference design all others are compared with (design 1).
Benchmark	A fixed test program with known inputs and answers.
Bottleneck	A part of the work taking a large share of the time.
Candidate	A design being compared against the baseline.
CPI	Cycles Per Instruction: cycles divided by instructions executed.
CPU	Central Processing Unit: the processor.
Cycle	One tick of the hardware clock in the simulator.
D register	A data storage slot in the Hack CPU.
Digital twin	Here: a stored, analysable record of a simulated design's behaviour.
Geometric mean	An average for ratios: multiply n values and take the n-th root.
GitHub	A website hosting public code repositories.
Hack	The simple 16-bit computer from the nand2tetris course.
Halt	Reaching the program's final infinite loop.
Hash / fingerprint	A short code computed from content; changes when the content changes.
HDL	Hardware Description Language: text describing how hardware is wired.
ISA	Instruction Set Architecture: the rules for what each instruction does.
Jump folding	HackTwin design trick: skipping a repeated jump instruction.
Lineage	Which design a variant was derived from.
MAE	Mean Absolute Error: average size of prediction errors.
Machine learning	Programs that learn patterns from examples.
Netlist	The list of parts and the wires between them.
PC	Program Counter: address of the current instruction.
R^2	A score comparing a model with guessing the average (1 perfect, below 0 worse than average).
RAM	Random-Access Memory: the data memory.
RMSE	Root Mean Squared Error: like MAE but larger errors count more.
ROM	Read-Only Memory: holds the program.
Simulator	Software that imitates hardware described in HDL.

SQLite	A database stored in one file.
Speedup	Baseline cycles divided by candidate cycles.
Trace	A cycle-by-cycle record of the machine's signals.

Conclusion

HackTwin turns raw nand2tetris simulator output into a checked, stored and comparable record of each Hack CPU design. With it we can say, with evidence, which designs are correct and which use fewer simulated cycles, without claiming anything about real hardware speed, area or power.